

From business processes to software services and vice versa—an improved transition through service-oriented requirements engineering

Sebastian Adam^{*,†}, Norman Riegel, Joerg Doerr, Oezguer Uenalan and Daniel Kerkow

Fraunhofer IESE, Fraunhofer Platz 167663 Kaiserslautern, Germany

SUMMARY

Business processes need to be agile and flexible to help organizations stay competitive. For this purpose, SOA promises the reuse of already existing information system functionality for enabling enterprises to change the business processes more quickly. However, many promises made by SOA authors have not found their way into practice, because methodological guidance that constructively assures a more successful SOA application is still missing. In this article, service-oriented requirements engineering is therefore introduced as a discipline aiming at a better and more systematic handling and alignment of SOA and Business Process Management. Our industrial experience in this regard is also described. Copyright © 2011 John Wiley & Sons, Ltd.

Received 20 June 2011; Accepted 21 June 2011

KEY WORDS: transition; service-oriented; processes

1. INTRODUCTION

1.1. Context

Business processes have become increasingly important in many enterprises. They determine the procedure for developing and distributing value for the customers and are key drivers for the three success criteria: cost, quality, and time.

Enterprises remaining successful in the market have to deal with changing requirements such as new market demands, competition, or regulations and compliance. For this purpose, Business Process Management (BPM) [1] recommends a set of activities for managing and improving business processes continuously. BPM covers all phases of the business process life cycle in order to provide a holistic support. Thus, the phases of design, deployment, and operation/evaluation are supported. In particular, BPM has to assure the efficiency and flexibility of business processes in order to meet the aforementioned success criteria. Without these properties, an enterprise will not be able to stay competitive on the market.

Apparently, flexibility and efficiency are strongly dependent on the information technology (IT) used. Although IT is not a guarantee for any competitive advantage, having no IT will surely cause business failure [2]. Therefore, IT and especially information systems are often considered an enabler for market success. With regard to BPM, IT should support all phases of the business process life cycle in a manner that is as holistic as possible. However, whereas design, simulation, and evaluation of a process are performed by business people (using IT, of course), implementation and execution are pure

^{*}Correspondence to: Sebastian Adam, Fraunhofer IESE, Fraunhofer Platz 1, 67663 Kaiserslautern, Germany.

[†]E-mail: sebastian.adam@iese.fraunhofer.de

IT-related tasks, as business people usually do not have the required technical knowledge. The resulting separation between the business view and the IT view on business processes significantly complicates, in practice, an 'ideal' BPM. The so-called business-IT gap (see line in Figure 1) thus leads to costly and time-consuming human intervention when business process models are to be brought into production or vice versa.

Because the transition from the design to the deployment phase is a central one with regard to enterprise flexibility, the vision of 'business process automation' aims at lowering the effort for developing and releasing information systems from business process models by automating this transition (see dotted arrow in Figure 1). Even if there is still much work to do, several vendors such as IBM, Software AG, Progress, Oracle, and others have therefore started to support this vision through the provision of full-fledged BPM suites. In this context, the paradigm of Service Oriented Architecture (SOA) [3] is considered as a powerful partner for BPM, providing mechanisms for making information systems and thus enterprises more agile and flexible.

There are several definitions of SOA today. Some references consider SOA as an IT paradigm; others define SOA as a holistic approach already integrating BPM aspects. For most enterprises, however, SOA is just a technical issue [4], even if its benefits are primarily concerned with business. The basic idea of SOA is the restructuring of information systems or IT landscapes into loosely coupled, independent software services. These software services, which can be recursively integrated into more coarse-grained services, should allow the reuse of existing functionality in order to shorten the time between process design and system implementation when business requirements change.

In SOA, the development is therefore typically separated into service engineering and application engineering (also termed as 'programming in the small' and 'programming in the large' [5]). The former describes the development of reusable software services, whereas the latter defines the development of information systems based on service reuse.

In this context, a visionary view is often published, stating that business process modeling, and thus more or less non-technical work, should be sufficient for application engineering. Some references therefore consider SOA as a next step in abstraction, enabling 'business process-oriented development' [5].

1.2. Problem statement

The assumption behind SOA that new information systems can be developed by transforming business process models automatically into service orchestrations is still a dream [6], even if corresponding promises have already been made. Besides the fact that, due to a lack of semantics, business processes have still to be translated manually into an executable format, we consider the insufficient reuse support in most SOA approaches as the most important problem. The main challenges in developing service-oriented information systems is therefore an insufficient methodology for the refinement and mapping of business processes to existing service infrastructures, even if several vendors already provide initial tool support in this regard.

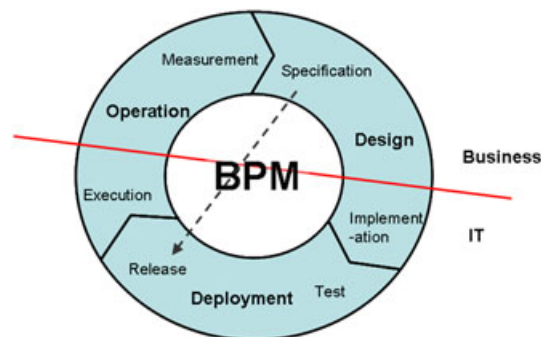


Figure 1. Business process life cycle and the gap between business and IT.

The reasons are obvious: On the one hand, fine-grained software services are close to the technical implementation and thus not applicable for business analysts during business process design. To use them as business process automation units, business processes first have to be refined until a very fine-grained technical level is reached on which these services can be integrated to provide the required high-level business functionality (see Figure 2, left). Because of a lack of semantic information on state-of-the-practice business process models, this stepwise refinement can only be done by humans and is thus costly and time-consuming. However, even the ongoing research endeavor on semantic services and semantic business processes does not seem to be a reliable contribution, as until now, it is completely unclear whether the additional (semantic) specification effort has a justifiable return on investment (at least within only one enterprise).

On the other hand, coarse-grained services promise to be easier to apply during business process design. They are much closer to the business perspective and could be linked ‘as-is’ with the processes they support (see Figure 2, middle (arrow)). Unfortunately, this is only true in theory. In practice, however, the lack of systematic reuse often results in a mismatch between the business process models and the available services; that is, the existing services do not immediately fit the requirements elicited during business process design. From a service engineering point of view, this is caused by the fact that the service usage within changing contexts is often neglected, resulting in too specific, poorly reusable services. However, also from the application engineering point of view, effective reuse is often hampered, as the actual capabilities of the service landscape are typically not taken into consideration. This leads to the situation that existing services are poorly reused, even if they would be appropriate candidates for supporting a certain business process. Services are therefore often modified when they are to be (re)used, or new services are developed. This problem is quite similar to reuse problems already known from component-based software development (see Ref. [7]).

Hence, without a systematic approach, the business–IT gap will not automatically be bridged by using SOA in BPM; that is, SOA will not automatically provide benefits for business agility. Indeed, approaches such as that of Wahli *et al.* [8] explain how ‘good’ services could be identified based on business process requirements, existing assets, or both. However, identifying services in this way does not automatically assure that they are also usable when they are to be applied in other processes or contexts. Consequently, the question of how services should be provided in order to be really usable in different business process contexts has not been answered satisfactorily until now.

1.3. Contribution

To cope with this challenge and ultimately improve the integration of BPM and SOA, we are convinced that BPM must be closer aligned with the actual SOA landscape of an enterprise in a systematic manner. In our opinion, this can only be achieved by an integrated engineering approach that provides services on the right level of abstraction and reusability, and thus improves the transition between the process design and deployment phase significantly. Even if tool support is inevitable in this regard, a systematic methodology is the foundation.

This paper extends our previous work on the integration of BPM and SOA by providing more details on the methods developed, by integrating them in a common framework and by validating them in two case studies. The contribution of this article is therefore as follows: In the next section, Service-Oriented Requirements Engineering (SORE) is introduced as a discipline aiming at aligning

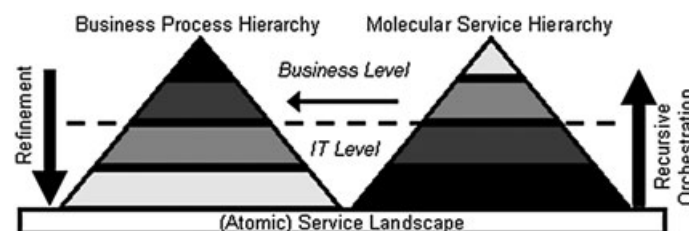


Figure 2. Business process and molecular service hierarchy.

BPM and SOA more explicitly. For this purpose, we integrated our (thus far isolated) methods and concepts described in Refs. [9,10], for instance, into a holistic method.

In subsequent chapters, we then describe both a method for developing reusable software services based on a representative set of business processes to be supported and a method for developing new business process applications under the early consideration of existing assets. In this regard, our industrial experience with these approaches will also be described. The article closes with a conclusion and an outlook on future work.

2. INTERGRATING BPM AND SOA

Experience has shown that ad hoc approaches are often not able to assure repeatable success in developing software, which meets the needs of a business department, in a reuse-based way. This observation also seems to be true when software development is just a small part of a higher-level concept such as BPM.

For improving service reuse, Lee *et al.* [11] therefore suggest applying systematic product line engineering (PLE) on SOA. In general, PLE [12,13] splits the development activities into two phases: family engineering and application engineering (see Figure 3). The purpose of family engineering is to systematically build up a reuse infrastructure from which products of the (product) family can be developed. The actual development of these final products is then performed during application engineering by explicitly considering and integrating the assets of the reuse infrastructure.

The main concept of PLE with regard to reuse is the systematic (mostly proactive) definition of all product variants that should be developed based on the family's reuse infrastructure. For that purpose, common and variable characteristics within the family are planned systematically. For instance, developing a set of different domain-specific project management software could be significantly improved by systematically defining in advance the common and variable functions. For software developing enterprises, product lines have been proven to significantly increase development efficiency.

However, product line approaches cannot be transferred to the BPM/SOA context without adaptation. The most important difference we see in this context is the shift from software developing enterprises to software applying organizations. Whereas software developing enterprises are mostly specialized on a certain set of software products and could therefore define a clear product family (domain scope), enterprises, which just use software, use a multitude of completely different systems such as accounting, product planning, construction, office applications, etc. (called 'system of systems'). Consequently, identifying a product family based on a common functionality between these systems is nearly impossible.

At this point, the role of BPM and SOA comes into play: Since one goal of SOA in BPM is the integration of different systems in a process-oriented way, using business processes as drivers for family engineering seems to be more suitable in this context. In fact, there are approaches such as that of Bayer *et al.* [14] that transfer concepts of product families into process families. The idea is to define all variations and commonalities within a set of business processes and to develop a related reuse infrastructure for developing the process-supporting information systems.

However, in the BPM context, we consider PLE including the modeling of all process variants and supporting services in advance as generally applicable. First, enterprises usually have to react to

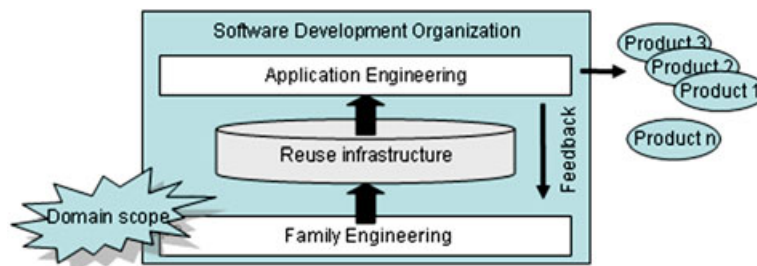


Figure 3. Overview of product line engineering.

unforeseen changes. Thus, modeling all process variants that might occur sometime in the future is just impossible. Second, reuse in an enterprise context should support quick changes without sacrificing the uniqueness of the organization, because enterprises must remain different in order to realize their unique selling points. Hence, adhering to predefined reference process variants is not an appropriate means for business. Third, improving the IT is never an end in itself. Consequently, there must always be quick wins when investment in better IT infrastructures is to be justified.

Nevertheless, we are convinced that an integrated BPM/SOA engineering approach that applies the key concepts of software product lines is a successful means for achieving closer and faster alignment between process design and deployment within the business process life cycle. Especially the early consideration of existing assets during the development of new information systems, as proposed by product line application engineering, seems to be an appropriate means for ensuring that business process requirements and services fit together better.

As a concrete approach for achieving a closer alignment of the disciplines of application engineering and services engineering, we consider SORE as an appropriate mediator. Basically, SORE is a still evolving [15] discipline that deals with the identification of services for reuse, as well as with the reconciliation of existing service capabilities and application requirements during reuse [9]. In order to emphasize the role we see for SORE in the strongly business-influenced SOA context, we have developed a BPM/SOA model that integrates some concepts of BPM, SOA, and PLE (see Figure 4).

As it is quite common, also this model separates the development of SOA systems into service engineering and application engineering. The former is responsible for developing reusable services and deploying them into the service landscape. Even if this looks quite similar to family engineering in PLE, the main difference is that service engineering does not provide the complete reuse infrastructure that could be instantiated, but only loosely coupled reuse assets in terms of software services. Furthermore, the service landscapes may also contain services that have been delivered from other sources such as external providers or legacy systems. We therefore see the purpose of service engineering primarily in the provision of software services on an appropriate level of abstraction. On the basis of these business services, application engineering then defines, implements, tests, deploys, and validates business process applications according to the BPM life cycle.

The role of SORE consists of mediating between application engineering and service engineering in order to achieve a good fit between business needs and available services. Therefore, SORE should provide support for

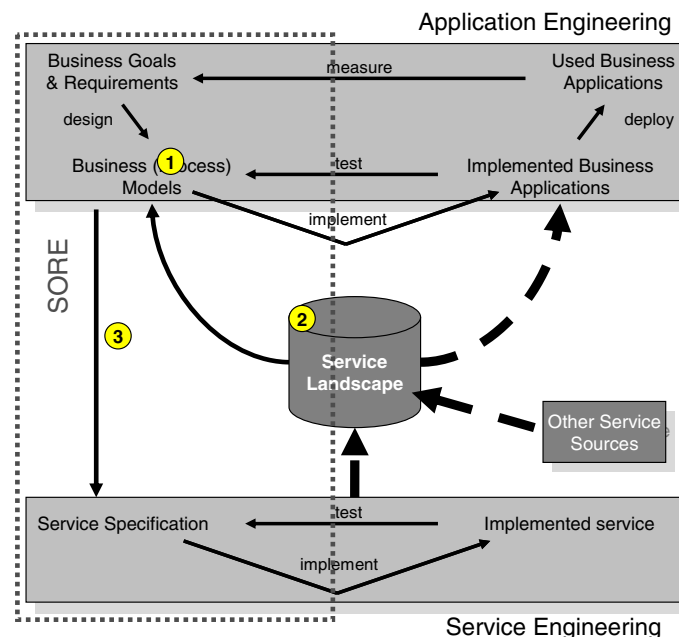


Figure 4. Integrated view of BPM, SOA, and PLE.

- (1) *the process of reconciliation* of business needs with available services in order to support the achievement of both business goals and high reuse;
- (2) *the appropriate representation* of actual service capabilities to business people in order to make these capabilities explicitly visible already during an early project phase; and
- (3) *the process of defining services*, that is, the determination and clustering of (innovative) features a service should provide in order to also be beneficial in future contexts.

Consequently, the provision of service capabilities on the right level of abstraction and granularity is a crucial aspect when it comes to really benefiting from SOA. As far as we can tell, this has not been solved satisfactorily yet, even though a multitude of service-oriented development approaches, such as that of Wahli *et al.* [8], are currently emerging. In this context, the notion of SOA governance has also to be mentioned. However, as this term is still fuzzy and often used for marketing purposes, we are going to avoid it in the rest of this article.

In the next two sections, we present our method for the process of defining services (3) and the process of reconciliation (1).

3. THE PROCESS OF DEFINING SERVICES

One of the possibilities provided by SOA is the ability to decompose monolithic and complex software systems into independent function blocks that can be distributed separately in terms of software services via the network. The ability of business departments to use certain functions ‘on demand’ and seamlessly integrate these services into their business processes enables more flexible and individual solutions.

Of course, such a reusability of software services in different business processes mainly depends on the context for which these services have been designed. However, in addition, it is also a challenge to identify these services on the right level of abstraction and in the right functional range. In this context, functional range is defined as the amount of different functions provided by a software service (for a certain context), whereas the level of abstraction describes whether a function realizes a more business-oriented function or a more technical-oriented function. Thus, organizations have to find out how to restructure their IT according to these dimensions (see Figure 5).

On the one hand, when the functional range of a software service is large, the functionality might be too overloaded for the actual needs of most business departments, and thus might not be considered appropriate. The same holds true when functionality is provided on a very high level of abstraction. In this case, the service might contain a functionality that is too specific for a particular business process and, therefore, not well reusable.

On the other hand, when the functional range of a service is quite small (e.g. only one function from a set of related functions is provided), most business departments would probably not be willing to invest integration effort for them. Again, the same holds true for a very low level of abstraction, that is, functions that do not provide an immediate business support.

In SOA, the problem of feature identification is therefore different from those in ‘normal’ software products, and has not been solved satisfactorily yet. A requirements engineering method identifying common and different needs of the business departments is therefore a required means for finding

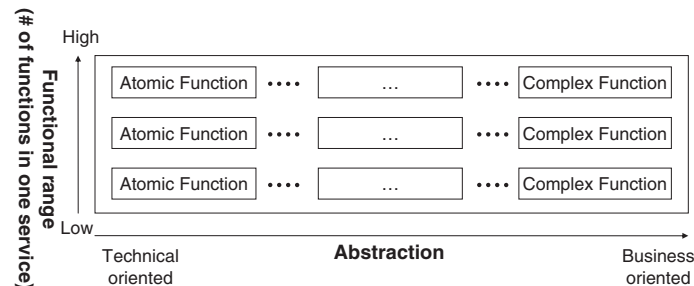


Figure 5. Abstraction and functional range of services.

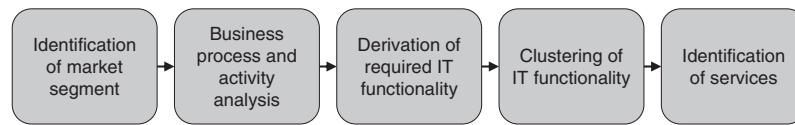


Figure 6. Overview of the method.

the ‘right’ functional range and abstraction level for successful services. To meet the ‘right’ level of abstraction and functional range, we propose a method (see Figure 6) based on business process-driven requirements engineering with variability and correlation analysis techniques.

3.1. Identification of business segment

The first step of the method is the clear identification of the business segment for which software services should be provided. Defining a clear segment scope is needed in order to prevent the effort for analyzing the segment from exploding. However, we do not focus on this aspect in the article, and assume that a certain segment has already been defined.

3.2. Business process and activity analysis

Since it is very costly to model the complete business, only a set of representative business departments from this business segment are invited and involved in the analysis of corresponding business processes and activities. Typically, business departments to which a good and trustful relationship already exists are preferred candidates for involvement. Together with people from these departments, the business processes in the scope of interest (e.g. personnel administration) can then be analyzed and specified.

Because it is, in practice, often a challenge to model and analyze the processes carefully, our method provides a detailed elicitation checklist (see Figure 8 for an exemplary extract) to ensure that the processes are analyzed systematically on a common level of abstraction according to our meta model. The checklist supports specifying business processes with related nonfunctional requirements (NFRs) on the level of elementary activities in order to directly derive a complete set of service functions.¹ In this context, it is required that each elementary activity within the process models is described in more detail; that is, working instructions on how a process stakeholder should interact with a system must be available. We propose describing the elementary activities using a minimized Use Case [16] template (see Table I) to gather all alternating interaction steps.

3.3. Derivation of required service functionality

On the basis of the set of all activity descriptions from the processes of one representative business department, the required service functionality can then be derived. Each system step within the set of activity descriptions is extracted and transformed into a service function description (see Table II). To specify the function’s signature, that is, the list of input and output parameters, embedding of this function within the originating activity description is taken into consideration. Thus, the parameters are first selected based on the usage within a dedicated activity.

However, because one important characteristic of a software service is its context independence, the functions must not be specific for a certain context of use. To realize this, all extracted service functions with the same functionality have to be consolidated, including the elimination of synonyms. During this consolidation, the input and output parameters are generalized (i.e. the most generic signature is taken) to make a function applicable in as many usage contexts as possible. Thus, only the most generic signature should be kept.² This also holds true for the NFRs: Only the most restrictive NFRs of the same type should find their way into the final function description. In

¹An elementary business activity is ‘an atomic step within a business process that ends in a persistent state’.

²We abstract in this method on how an alternative invocation with different input and output parameters will be realized technically.

Table I. Example of activity description

Activity description	
Name	Update personnel data
Flow of events	1. The actor enters the name of an employee from which the personnel data should be updated. 2. The system identifies and displays a list of employees (name, department) that match the query. [NFR: response time <5s.] 3. The actor selects one employee from the list. 4. The system displays the personnel data of the employee. [NFR: response time <5s.] 5. The actor updates and saves the data. 6. The system updates the data in the database. [NFR: response time <5s.]

Table II. Example of service function description.

Service function description	
Name	Retrieve employee
Description	This function retrieves a list of employees that match the query.
Input	Name
Output	LIST OF (id, Name, Department)
NFR	Response time <5s.

doing so, a list of disjunctive service functions required to support all considered department-specific business processes is developed.

3.4. Clustering of service functionality

As a software service should be cohesive, it should typically provide a set of related functions. On the basis of the list of functions identified before, a second clustering step in which logical function blocks are built is therefore necessary. Typically, function blocks are developed based on the type of functionality they provide or the data structure on which they operate. We recommend combining both approaches and applying both clustering strategies depending on whether a related backend data structure exists for a certain function or not. For instance, all functions modifying or retrieving a specific entity (CRUD-operations) could be integrated within a function block ‘related functions’, whereas all pure processing functions that do not operate on a specific data structure could be integrated based on the logical type of their functionality (e.g. calculating functions).

3.5. Identification of services

The function blocks identified thus far just describe an appropriate solution for one specific business department, as all functions have been derived from its concrete business processes. In order to derive software services that can be used by as many business departments as possible, the function blocks of all involved representative business departments have to be generalized using a variability and correlation analysis.

The step for identifying the services is therefore structured as follows: Function map creation, Function map optimization, Service derivation, Service specification.

Function map creation

In order to get a good overview of the required common and variable functions, we propose the adaption of the method described by Schmid [17]. The main part of this method is the development and optimization of a product map, which we call function map in our context.

The function map (see Table III) consists of the whole list of identified functions (2nd column) from the representative business departments clustered by a consolidated set of the logical function blocks

Table III. Example of function map.

Logical function block	Service functions	Business department	Business department	Business department	Business department
Employee related functions	Create employee	x	x	x	x
	Delete employee	x		x	x
	Retrieve employee	x	x	x	x
	Update employee	x	x		x
Business trip related functions	Plan business trip	x		x	
	Account business trip	x		x	
	Update business trip	x			

(1st column). Of course, synonyms must be eliminated, if applicable. For every business department that has been analyzed, an individual column exists in the table (3rd to nth column). If a specific function has been identified for a certain business department, the corresponding cell is marked.

Function map optimization

After creating the table, outliers must be detected and removed. In the context of the function map, we consider as outliers those cells of a certain business department that are apparently different from most other cells in the same row (e.g. 'Delete employee', business department 2; Table III). These outliers must be removed in order not to affect the correct allocation of functions to final services. For optimizing the map, we suggest the following heuristics:

H1. When a business department does not use a specific function but most other business departments do, it has to be decided whether this business department could also benefit from this function. If a real benefit is expected, the corresponding cell is also marked. If there are good reasons for not adding this function, the cell remains empty.

H2. When a business department uses only a few functions within a logical, rather specific group (i.e. most business department do not use any function from this group), it has to be analyzed whether this choice is absolutely necessary for him. If there are no good arguments that the business department absolutely needs this function, the corresponding mark should be removed from the cell. Otherwise, the mark should be kept, of course.

Following these rules, for instance, the empty cells of business department 3 are marked because it is expected to also benefit from both functions (see Table IV).

Service derivation

After the function map has been optimized, services are derived. Software services should be reused 'as-is' without any specific configurations for certain business department needs. Consequently, the provided functionality has to always remain the same, and department-specific solutions should be built by integrating services in a variable way [18] instead of making the services themselves variable.

Table IV. Example of optimized function map.

Logical function block	Service functions	Business department	Business department	Business department	Business department
Employee related functions	Create employee	x	x	x	x
	Delete employee	x		x	x
	Retrieve employee	x	x	x	x
	Update employee	x	x	x	x
Business trip related functions	Plan business trip	x		x	
	Account business trip	x		x	
	Update business trip	x		x	

Therefore, our approach does not propose defining variable and common functions within a software service, but to use the analyzed variability in the demand for the distinction of services.

As the logical function blocks were identified based on high cohesion between the functions, it appears reasonable to make each function block a service. However, as this logical clustering was just based on expert knowledge, there has not been any assurance yet that the demand for each function within a service will actually correlate. As a consequence, the risk that business departments do not require service functions (but have to pay for them) remains, which might lead to less demand for that service.

In order to give a comprehensible justification for the service derivation, we propose assessing each service based on the correlation between the demands of its functions. Therefore, we propose applying the ϕ -correlation analysis (see statistics literature such as Ref. [19]) on all nonidentical rows within the function map in a pairwise manner in order to measure whether there is a correlation between the demand for two functions within a service (i.e. between two dichotomic variables with the values 'required' and 'not required').

The calculation of the ϕ -correlation is described as follows: Let a be the number of business departments requiring function 1 and function 2, b the number of business departments not requiring function 1 but function 2, c the number of business departments requiring function 1 but not function 2, and d the number of business departments requiring neither function 1 nor function 2; let $n=a+b+c+d$ (all representative business departments).

Then:

$$\phi = \frac{a \cdot d - b \cdot c}{\sqrt{(a+b) \cdot (c+d) \cdot (a+c) \cdot (b+d)}}$$

For checking the significance³ of ϕ with $\alpha=0.05$ (i.e. with a probability of error below 5%), it has to be calculated whether

$$n \cdot \phi^2 > \chi^2_{(0.95,1)} = 3.84$$

On the basis of the two calculation results, we recommend proceeding as shown in Table V.⁴

For instance, $n=15$, $a=9$, $b=2$, $c=1$, and $d=3$. Then,

$$\begin{aligned} \phi &= \frac{25}{\sqrt{2200}} = 0.53 \\ n \cdot \phi^2 &= 15 \cdot 0.53^2 = 4.26 > \chi^2_{(0.95,1)} = 3.84 \end{aligned}$$

This means that the considered functions should be combined into one service because there is a significant correlation between the demands for both.

In order to avoid such complex calculations for every row, a quick test can be applied: If two rows within a logical function group are marked identically, the corresponding functions can be combined within a common service (e.g. 'plan business trip', 'account business trip', and 'update business trip' in Table IV). However, if two rows are not completely identical (e.g. 'create employee' and 'delete employee'), this quick test is not applicable for getting reliable results.

Service specification

Finally, for all functions integrated in a service, the most generic signature has to be defined again, resulting in service specification. In order to provide these capabilities on an appropriate level of granularity, we therefore recommend clustering the service functions into so-called conceptual services [9],

³If the number of involved representative customers is less than four, a significant result can never be reached with $\alpha=0.05$.

⁴ ± 0.4 is just a first starting point and has to be adapted based on experience gathered during future application of the method.

Table V. Decision table for service derivation.

φ	$n \cdot \varphi^2$	
>0.4	>3.84	The demands for the two functions strongly correlate. It makes sense to provide both within a common service.
<-0.4	>3.84	The demands for the two functions contradict. Business departments requiring function 1 do not require function 2, or vice versa. It does not make sense to provide both functions within a common service. Distinct services for both functions should be defined.
≤ 0.4 AND ≥ -0.4	>3.84	It is unclear whether the two functions (strongly) correlate or contradict. A decision for or against the definition of distinct services should be made by an expert.
	≤ 3.84	Involve more representative business departments or let an expert decide for or against the definition of distinct services.

which, in our experience, are perfect candidates for communicating IT capabilities to business people owing to their logical capsulation. Conceptual services are abstract descriptions of capabilities and system functionalities that are provided by the existing service landscape. However, conceptual services do not directly represent real services that are executable. This abstraction enables the separation of concerns for the business analysts that only need to know which functionality is basically available without the need to know what technical interfaces the real services have. This abstraction from the real technical services and the focus on understandability for humans rather than on interpretability for machines are an advantage especially in requirements negotiation [20]. In particular, business people can better understand what is easily feasible with the existing services, whereas developers can assess much faster how well their reuse infrastructure already covers the mentioned needs. In the following discussion, a template for a conceptual service is presented (see Table VI), where each field is described in detail.

With regard to the NFRs, we propose using the business department-specific, function-related ones to define different service-level agreements (SLAs) on which the consolidated services could be provided. For instance, the most restrictive NFRs within a service mentioned by the representative business departments could be combined into an SLA 'highest quality', which could then probably allow selling the service more expensive than with an SLA including the less restrictive NFRs. However, how to define these SLAs is not addressed in detail in this article.

4. THE PROCESS OF RECONCILIATION

Although services have been derived in a systematic manner as mentioned in the previous section, they do not automatically lead to a tighter fit. A reason is that business stakeholders are often not aware

Table VI. Template for conceptual service description.

Service template (S-ID)		
Purpose	Purpose of the service	E-mail address validation service
Classification	Branch: Branch of the service, e.g. information technology Domain: Domain of the service, e.g. user management	Communication
Functions	<u>General CRUD functions</u> List of functions in order to create, read, update, and delete items, e.g. create, read update, and delete users <u>Special functions</u> List of functions that are non-CRUD functions, e.g. login user	Validate e-mail address
Screens	Pictures of mockups screens that represent the service	
Assumptions	Assumption for the usage of the service, e.g. users are all enterprise employees	Only new e-mail addresses need to be validated
Additional benefit	Additional benefits of the service, e.g. user management of multiple users at once.	Security

whether their requirements are feasible with a set of given services. Requirements must therefore also 'depend on existing system capabilities and not just on what stakeholders believe they need' [21]. Several references (e.g. Refs. [22,23]) have therefore stressed that the capabilities of a reuse infrastructure must be already taken into consideration during the early requirements phase; otherwise, a high degree of reuse will not occur.

The major challenge to be managed by requirements engineers and business analysts is therefore to satisfy and reconcile actual needs with a given service infrastructure. Hence, customer expectations must be negotiated against the capabilities and constraints of the existing services [24,25], and proactively refined into business process application requirements that are actually realizable within the estimated time and budget.

In this section, we present an approach for service capability-based business process design that allows developing business processes and supporting business process application with the early consideration of an existing service infrastructure in a cooperative manner.

As Tsai *et al.* [15] highlight the need for a domain-specific, framework-oriented analysis in SORE, we also propose a similar approach for this aim. However, whereas traditional framework-oriented analysis approaches, such as the approach of Zhu and Tsai [26], use real technical framework components, our approach makes use of the more abstract conceptual services belonging to a certain business segment. The reason is that business requirements can only be reconciled with service capabilities in the early phase of business process design when both are described on a level of abstraction that is understandable and comparable for business people. The basic steps of our method are depicted in Figure 7.

4.1. Identify and involve stakeholders

According to this method, the first step is the identification of relevant stakeholders in the business department for which a business process application should be developed and their involvement in the elicitation process. Getting access to all 'right' stakeholders is a crucial issue, and usually requires the support of the (top) management.

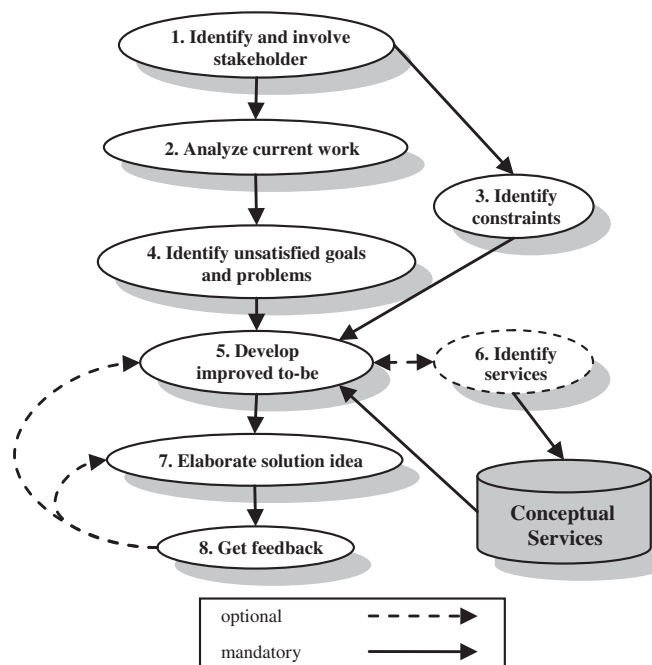


Figure 7. Method overview.

4.2. Analyze current work

With the stakeholders identified before, the current business processes to be supported by IT are systematically analyzed and modeled in joint workshops in order to get a common picture. In order to quickly obtain this comprehensive overview of the current business processes, a systematic, algorithmic-like elicitation checklist is used. This checklist (see Figure 8) ensures that a process model can be sketched already during the workshops, enabling the participants to directly make comments and corrections. Thus, misunderstandings, incorrectness, or incompleteness can be avoided right from the start. Furthermore, the business processes can be systematically elicited on a common level of abstraction; in our case, the level of elementary business activities.

4.3. Identify unsatisfied goals and problems

On the basis of the elicited business processes, unsatisfied goals and problems that currently hamper more efficient and satisfactory work are therefore systematically analyzed in a next step. The problem analysis is performed by iterating over each business activity and each activity sequence within the business processes modeled before. By posing related questions, typical problems in business processes such as media disruption or an insufficient degree of automation or parallelization can be identified systematically. Furthermore, based on this problem analysis, first conceptual services can be preselected as possible solution candidates.

4.4. Identify constraints

In order to ensure that all relevant information with regard to given constraints is also taken into consideration, the constraints posed by regulation and policies are systematically elicited from the responsible persons using a similar question- and checklist-driven approach.

4.5. Develop improved to-be processes

The constraints, current business processes, and identified problems and unsatisfied goals are then taken as input for cooperatively developing ideas for improvement, and thus improved to-be business processes. Hence, the IT department collaborates with the involved stakeholders in joint workshops in order to find a suitable solution that solves the department's problems. Thereby, the conceptual services come into play. Every conceptual service is presented by giving a short summary of its general purpose and functionality. Furthermore, based on the problem identification and the service preselection, the specific benefits of a certain service are motivated.

Then, together with the stakeholders, the conceptual services are annotated to corresponding activities in the processes (see Figure 10 (left) for an example). In doing so, the stakeholders easily get an understanding of which business activities can basically be supported by available services. Furthermore, they can discuss how this conceptual service, and thus which conceptual service functions, would concretely be used within each business activity. Hence, they can check the applicability of each service and make remarks on whether adaptations or extensions are needed.

- ...
- Which tasks / activities are performed in order to fulfill the responsibilities?
- For each task / activity:
 - Which material / devices / systems / etc. are used to perform this task?
 - Which preconditions must be fulfilled that the task can be started?
 - ...
 - Which information / input is needed for this task?
 - For each input:
 - Who/ what delivers the input?
 - Which information / output is produced in this task?
 - ...
 - How often is this task performed?
 - ...

Figure 8. Excerpt from an elicitation checklist.

When no conceptual service can be annotated to a business activity or when a conceptual service function is missing that would be needed to perform an activity, the following resolution strategy is applied. First, it is checked whether the required functionality is provided by another conceptual service, maybe even in a service framework belonging to another business segment. If no appropriate functionality is found, the IT department and stakeholder cooperatively discuss whether the activity (or even the entire process) could be changed in a way that existing services can be used, with the goal of the activity/process fulfilled nevertheless. Only if this is not possible must new services be defined or existing services be modified. In this way, missing functionality that cannot be solved by the existing services can be revealed early and can be prepared for future usage.

4.6. Elaborate solution idea and get feedback

After these cooperative steps, the IT department elaborates the solution idea by resolving the conceptual services (i.e. they develop a prototype). Then, a coherent concept is presented to the business department to get feedback and make final adaptations, if necessary. This feedback from stakeholders who are affected by the discussed changes is very important. By getting feedback, the IT department can get an impression of whether its ideas are on the right track and are considered helpful for the organization's intended purpose. Depending on the feedback result, some steps of the method have to be repeated.

5. EXPERIENCE AND DISCUSSION

5.1. Case study application

We have applied parts of our method in two real projects so far. In the first project, one of our industrial customers was interested in the extension of his product portfolio with 'add-on' application services. In the second case, we applied the method in a project for improving IT in public safety organizations. This case study is described in the following text.

During the first phase of the project, we applied the method explained in Section 3. In the project, the business segment was already given; thus, we only marginally conducted the 'identification of business segment' step. We then elicited processes by describing five large incident management scenarios of five public safety organizations, such as fire departments or police stations. After having specified the scenarios, we derived 29 (project relevant) activity descriptions by analyzing the activities in the processes in detail. These steps were mainly performed in group interviews or workshops, but also partly in individual interviews. On the basis of the activity descriptions, we derived a set of conceptual services that were used as input for developing real technical services ('development for reuse').

Despite the fact that we were looking at a very large and fuzzy-bounded application domain, we were able to derive a manageable number of conceptual services in this way (see Figure 9). The platform functionality included around 170 functions, which were then encapsulated in 17 conceptual service descriptions.

Then, during the second phase of the project, the aim was to evaluate the appropriateness of the developed service framework for the purpose of 'development with reuse', that is, to check whether the developed services were sufficient for addressing the needs of as many public safety organizations as possible. In order to perform this evaluation, we applied the method described in Section 4 several times with stakeholders from representative organizations, such as fire departments and police organizations, which had not yet been involved in the first project phase.

After identifying a scenario about handling a large amount of injured people that was assessed as a very challenging scenario by the involved stakeholders, we started with the analysis of the currently foreseen reaction (business processes) to this scenario. By using our method, we quickly came up with a corresponding business process model and could easily analyze unsatisfied goals, constraints, and possible problems to get a first impression of which conceptual services might provide significant support.

After that, we presented the set of conceptual services (just like in Figure 9) to the stakeholders, and we briefly explained each conceptual service one by one. We went through the whole scenario and

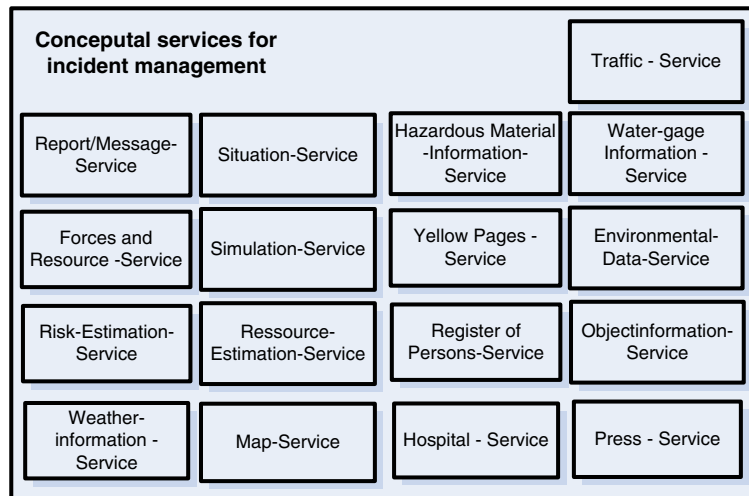


Figure 9. Framework of identified conceptual services in SoKNOS.

annotated cooperatively the business processes with our conceptual services (see Figure 10 (left) for an example using our own extension of EPC [27]).

After this annotation, we discussed each activity of the process with the stakeholders and how the functions of the conceptual services are actually used (see Figure 10 (right) for an example) in order to perform the activity. When a certain function was still missing, we looked at another service to see whether the required functionality was included there, or we discussed whether the process could be modified in such a way that the existing services fit. Of course, as we were still in the early phase of the research project; a couple of functions were still missing and thus could not be taken from the reuse infrastructure.

5.2. Lessons learned

Even if the results were overall positive, we gathered a lot of experience, which will guide our future work for improving of our method.

Identification of business segment (for defining services)

In our two projects, we identified the segments only on problem areas explicitly mentioned by the customers, without carefully analyzing the boundaries. We therefore encountered the problem of not

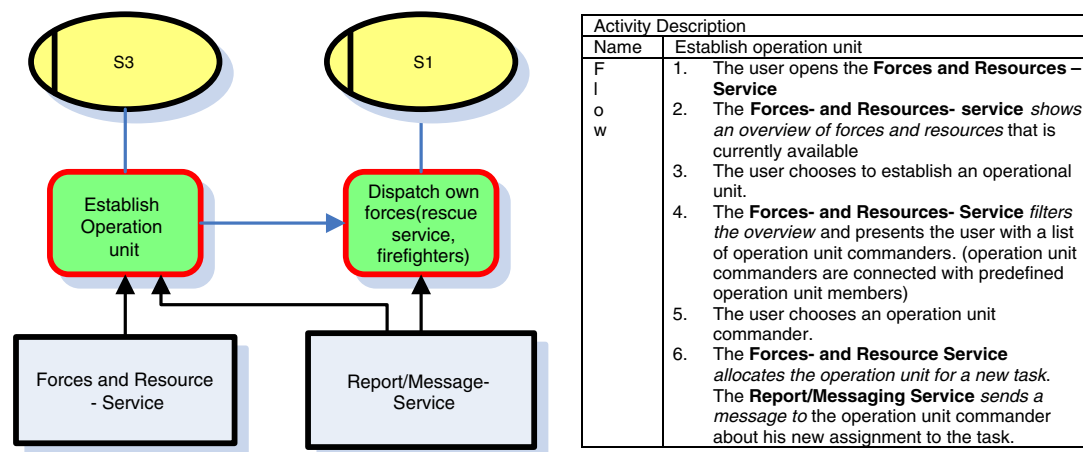


Figure 10. Part of business process with annotated conceptual services and activity description.

having any chance to check whether the set of elicited business processes was complete and covered the whole segment.

Thus, we found that the identification of business segment should not be done ad hoc and that systematic scoping, also with regard to economic aspects, is absolutely necessary. In this context, it is also important to explicitly list the business processes that should be taken into consideration. As future work, we therefore plan to elaborate and formalize this first step.

Business process and activity analysis (for defining services)

During the method's application, the stakeholders were highly motivated to cooperate with the service developers. This was due to the fact that they were interested in bringing in as much of their requirements as possible because they expected to get a more appropriate standard solution, which would also probably be cheaper than individual software.

The business process analysis based on our checklists also worked well. We performed it in individual interviews as well as in group interviews, resulting in suitable process and activity descriptions. In this context, we found group interviews to be more appropriate for getting a consolidated picture because individual interviews are much more time-consuming and their results often contradict each other.

Furthermore, we found the involvement of all stakeholders to be a crucial issue. We got the impression that some stakeholders were able to explain all processes in all details, but the risk that these explanations were just a subjective view remained. We also observed that most as-is processes could benefit from standard business process improvement rules.

Currently, we are extending our method with regard to these weaknesses. In particular, we are integrating business process improvement rules, creativity techniques, and early scoping activities to optimizing this step.

Derivation of required service functionality (for defining services)

After analyzing the business processes and activities, the derivation of required service functionality based on the activity descriptions was 'straightforward'.

However, we found that it does not make sense to define the function signatures in a too detailed way, as many aspects can only be defined when taking technical issues into consideration. Thus, we recommend specifying only the basic input and output data.

Clustering of service functionality (for defining services)

Thus far, the clustering of functions into services has been based only on expert judgment or common sense. Even if this clustering worked well in our projects, we see the challenge that there is no repeatable guidance for the clustering process. Although the clustering of functions based on the data structures on which they operate might often be clear, the clustering of pure processing functions might be more challenging. We therefore see the necessity to investigate in our future work whether cluster analysis techniques with a set of representative business departments would deliver better results than expert judgment.

Identification of services (for defining services)

During the identification of the services, we observed that the separation of data structure-based logical function blocks in different services is quite seldom. This could be because data structure-based function blocks often consist of CRUD operations, most of which are required together. However, function blocks clustered by the type of functionality were found to be unstable and were quite often split into different services.

Furthermore, we learned that a significant correlation analysis requires a certain amount of representative business departments, and that with less than 10, reliable results cannot be expected in general.

Identify unsatisfied goals and constraints (for reconciliation)

We found that stakeholders usually have a good idea of what goals and problems are within a process. However, not only stakeholders that have an overview about all system-relevant processes but also stakeholders that actually perform a process activity in their every day work must be considered. Even if they are not available, they should at least be asked with a questionnaire about their current problems and wishes.

Develop improved to-be processes (for reconciliation)

By using the conceptual services, we found that the stakeholders were enabled to abstract from detailed requirements, such as user interface issues, and concentrate on the process itself during the definition of an improved to-be situation.

In particular, we were able to communicate with the stakeholders in a predefined scope, which helped us to guide the business process design as far as possible towards reusable functionality. Thus, the complex phase of requirements analysis could be simplified (just business process design), which streamlined the entire requirements process. In particular, the abstraction from detailed requirements prevented the stakeholders from posing requirements that were too restrictive.

Furthermore, we were able to adhere to the real business processes and did not need to introduce reference business processes in order to achieve reuse. We think that this is a significant advantage over other reuse approaches, especially in domains that require support for rather ad hoc processes than predictable production processes.

In general, we found that the degree of technical innovation is very limited when the aim is to develop in a reuse-based manner. However, innovation and creativity are also quite important in 'development with reuse' because solving unforeseen problems with a given reuse infrastructure is challenging. So far, we do not have systematic support for this aim.

Identify services (for reconciliation)

During this step, we found that the stakeholders were able to make sure that they understood the concept of each service and that they were able to identify missing services and functions quickly on their own.

Elaborate solution idea (for reconciliation)

The early matching of conceptual services to the selected processes was a good starting point for elaborating the solution idea. Identified conceptual services could be mapped to real services and other technical solutions. However, to make sure that the services meet the business requirements of the stakeholders, early prototypes or mock-ups should be used to get early feedback from the stakeholders. Since when stakeholders communicate about the software application, they often think in terms of the user interface, and not in abstract concepts as services, the conceptual and technical services could also be represented in such a way.

5.3. Further discussions

Since enterprises need to adapt their business processes quickly to changing requirements, the supporting SOA must provide sufficient flexibility. By abstracting from both the technical implementation and the usage within a concrete process, our conceptual services can serve as a mediator that helps to quickly align new or modified processes with existing technical services or vice versa. In particular, conceptual services have been defined in a rather abstract manner and scoped to fit the required business functionality in a certain domain in general. Thus, when either service implementations or business requirements on processes change, the mediating conceptual service descriptions need not be changed to the same degree. Rather, only the link between the conceptual services and the changed business process, and thus the changed technical service, has to be adapted. Of course, when fundamental changes are required to meet new business needs, also the conceptual services might be changed. Nevertheless, appropriate software change management is necessary to support each type of change.

Furthermore, existing technologies should be used to apply our method. For this purpose, service repositories can be used and the metadata of conceptual services could be described with service repository description languages such as UDDI, ebXML, or the Directory Services Markup Language.

All these technologies are valuable and implemented by large vendors, both at the technical and semantic levels.

In addition, the support for meta data can help to identify services more quickly. Categorization and semantic representation can help to find a service at the conceptual level (e.g. I need a service that validates an e-mail address $\rightarrow$ E-Mail validation service) and find it in the service repository. This could be a means to annotate business processes with conceptual service descriptions more easily.

However, tool support and, in particular, services repositories are still beyond the scope of the conceptual work described in this article.

6. RELATED WORK

Thorough knowledge of the features that a product must contain is a prerequisite to building successful products. Approaches describing the identification of valuable features do exist (e.g. Ref. [28]), but they are not concerned with the specifics of SOA as to which ‘products’ can be deployed on a lower functional range and abstraction.

Methods on how to develop services in this context are still rare. Indeed, methods exist (e.g. Ref. [29]), in which, for instance, services can be identified in a top-down way (based on business processes, goals, or information models), a bottom-up way (based on existing legacy systems), and a meet-in-the-middle (a combination of both) way. However, such approaches aim at developing services without considering variable demands. Thus, services can be very specific for the individual needs and be provided on nonunique levels of abstraction and functional range. As the purpose of our approach is to provide beneficial application services for different business departments, we therefore need an approach explicitly addressing the variable and common needs between them.

Developing products with variable and common features is the main idea behind PLE [13]. Especially the detection and selection of possible product variants (‘scoping’ [30]) is an important issue here. However, as software services should be reused ‘as-is’, we do not aim at developing services that may vary in their functionality but rather at enabling variable solutions through the alternative usage of software services. Thus, in contrast to Ref. [30], we use the concept of product maps just as a means for identifying independent services, while we consider product variation just by the absence or presence of services.

In general, the idea of using PLE for the identification of services is not completely new. Chen *et al.* [31], for instance, propose a method for deriving services from legacy applications. However, no guidance is given on how to come up with services on the right level of abstraction and functional range. Lee *et al.* [32] also propose the identification of services through PLE. Indeed, this approach supports finding services with a good functional range through a systematic feature analysis but does not give any guidelines on how to create the starting set of business functions that the intended business departments are actually interested in.

In order to derive such a set of functions in a systematic manner, task- or business process-driven requirements engineering can be applied. Our approach therefore integrates the RE approach proposed by Paech and Kohler [33], which aims at systematically assuring that all relevant information is actually elicited.

The actual reuse of existing assets such as software services can basically take place at any level of abstraction. In IS-COTS products such as ERP systems, for instance, reuse is typically achieved on the level of entire business processes through the configuration of predefined reference models [34]. Approaches aiming to apply PLE to the business process context also choose a similar approach (e.g. Refs. [14,35]). These approaches extend variant-rich reference processes with decision models and thereby allow an automatic instantiation of business processes and their service-based implementations. However, because of the enormous process variety in today’s organizations and the attempt to keep individuality in the way of work, the proactive prediction of all process variants to be supported is nearly impossible [10], except for specific domains.

The level of business activities is therefore another level on which reuse can be applied and basically has already been part of, for instance, the ARIS framework [36]. Indeed, service reuse at this level allows defining flexible business processes, as the order of activities is not predefined. However, this approach also has some shortcomings. First, how an activity is performed is already predefined. Second, we found

in trials with such a reuse approach that it is more time-consuming and challenging for business people to click their way to a process while scrolling through an activity repository than frankly modeling a process using their own terminology and mapping this process to existing components afterward.

The third level is the level of business functions (e.g. service functions), which is also the level basically addressed by SOA. This level provides the highest flexibility in building business process applications but generally requires the most effort for refining requirements into an executable solution. However, as far as we can tell, most approaches in this area, including those proposed by leading BPM/SOA solution vendors (see, for instance, Ref. [37]), share two main shortcomings. First, they require specifying the business processes on a very low level of abstraction; otherwise, an annotation of the real service functions is not possible. This, of course, leads to an enormous complexity of process models and the service infrastructure, as no appropriate means for abstraction exist. Second, most SOA approaches are based on the assumption that an organization's IT infrastructure has been decomposed completely in a service-oriented way, and that thus all required services are either actually available or very easy to (re)develop. As a consequence, most approaches still propose a pure top-down approach, that is, a specification of business processes without reconciliation between the resulting requirements and service capabilities. Although this might work in an in-house scenario, it is hard to apply in settings where IT departments work for several more or less independent business departments.

To cope with this reconciliation problem, only some initial work exists. Rolland and Kaabi [38], for instance, propose intentional services based on the MAP formalism to match high-level business requirements with service functionality. However, although this approach provides a clear algorithm on how to match both views, we consider the required specification effort and the notations used not justified for a lightweight communication of IT capabilities.

Approaches from COTS-based requirements engineering (see Ref. [39] for a comprehensive overview) aim at matching high-level requirements in terms of goals with product features to find the 'best matching' product. Our approach shares the idea that the requirements should remain on a high level of abstraction in order to prevent restrictions that are too severe. However, we are convinced that goals alone are not sufficient and that the detailed way of work must also be known. Wang *et al.*'s [40] work is probably the first reference that introduces the notion of domain services as an encapsulating means to mediate between business requirements and the capabilities of real technical services without the need to predefine the process flows in which the services are used. Although this idea is quite similar to our notion of conceptual services, the approach [40] gives no guidance on how the domain services are to be used for the actual reconciliation process between business needs and service capabilities. In this regard, our proposed method is strongly influenced by the key concepts of Doerr *et al.* [41], who also aim at using a framework of abstract components and a collaborative method for improving and defining new processes. Their approach and the approach presented in this article differed in context (requirements engineering process improvement versus business process evolution) and the fact that the former [41] consciously avoids explicit modeling of processes.

As described in Ref. [42], for example, our work also tries to align business and IT. It complements the idea of a business-led organizational approach (cf. e.g. Ref. [43]) by making use of SOA and BPM as an organizational paradigm. Thus, strategic alignment is fostered by a holistic view on SOA as a business paradigm.

Recent approaches [44] significantly contribute to the ease of use of BPM and SOA integration by providing a complete repository as well as a multi-layer modeling method that allows implementing information systems in a consistent and maintainable way.

7. CONCLUSION

In this article, we discussed the role of SOA in BPM and presented the practical challenges regarding an ideal interplay. We explained that an unsystematic SOA will not entail any business benefits, and what should be done for improvement.

We then discussed the role of SORE as a mediator between application engineering and service engineering, and pointed out the necessity of defining and reusing software services in a systematic way, since otherwise, an SOA will probably not entail any business benefits.

As a consequence, we presented a method based on business process-driven requirements engineering, variability consideration, and correlation analysis for systematically packaging service functionality. While the business process-driven elicitation allows finding service functions on an appropriate level of abstraction, the variability and correlation analysis helps to check whether it makes sense to package certain functions within a common service.

Furthermore, we presented an approach for service-aware business process design. The novelty of this approach lies in the fact that we embed reuse capabilities in terms of conceptual services into a systematic process for business process design and thereby allow achieving a high degree of reuse in an early phase.

Although we have not been able to evaluate our approach in a controlled experiment, and the lessons learned have shown that there is still some work to be done, the case study experience confirms that we are on the right track with our approach.

Our hypothesis to be checked in a next step is that this approach leads to a higher degree of reuse when developing business process applications, and thus to less effort for a similar degree of reuse, compared with other approaches that propose to first refine business processes and to annotate services afterward. Of course, the success of our proposed method also depends on the quality and completeness of the underlying service framework and the involvement of the right stakeholders. Furthermore, our method is still limited to mid-term or long-term business process evolution and not applicable for short-time or even run-time adaptations. Besides the need to discuss the business processes with a group of people, also the degree of automated transition between conceptual services and real implementations influences the addressable time scope. Thus far, there is only a conceptual link requiring developers to still manually map the conceptual services within a business process model to an executable description.

Besides this weakness, we have to elaborate the method into a complete requirements engineering approach that goes beyond the activity of business process design. Furthermore, we plan to combine a rather workflow-oriented system development approach with a highly automated product derivation approach known from the PLE area. Finally, we plan to investigate how development for reuse can also benefit from creativity techniques.

ACKNOWLEDGEMENT

The research described in this paper was carried out under the SoKNOS project (grant no. 01ISO7009) and the ADiWa project (grant no. 01IA08006F), both funded by the German Federal Ministry of Education and Research.

REFERENCES

1. Smith H, Fingar P. *Business Process Management—the Third Wave*. Meghan-Kiffer Press: Tampa, FL, 2003.
2. Carr N. *Does IT matter? Information Technology and the Corrosion of Competitive Advantage*. Harvard Business School Press: Cambridge, MA, 2004.
3. Erl T. *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall: Upper Saddle River, NJ, 2005.
4. Amadee AG. Studie SOA Check 2007—Trends im deutschen Markt. (Available from: <http://www.soa-check.net> last visit 10 February 2008.)
5. Emig C, Weissner J, Abeck S. Development of SOA-based software systems—an evolutionary programming approach. *Proceedings of AICT/ICIW 2006*. IEEE Computer Society, 2006.
6. Papazoglou M, Traverso P, Dustdar S, Leymann F, Krämer B. Service-oriented computing research roadmap. *Dagstuhl Seminar Proceedings* 05462, 2006.
7. Atkinson C, Muthig D. Enhancing component reusability through product line technology. *Proceedings of the 7th International Conference on Software Reuse, LNCS 2319*. Springer, 2002.
8. Wahli U et al. *Building SOA Solutions Using the Rational SDP*. IBM Redbook. (Available from: <http://www.redbooks.ibm.com> last visit 10 February 2008.)
9. Adam S, Doerr J. The Role of service abstraction and service variability and its impact on requirements engineering for service-oriented systems. *Proceedings of the 32nd IEEE International Computer Software and Applications Conference*. IEEE, 2008.
10. Adam S, Doerr J. How to better align BPM & SOA—ideas on improving the transition between process design and deployment. *Proceedings of 9th Workshop on Business Process Modeling, Development and Support, Montpellier*, 2008.
11. Lee J, Muthig D, Naab M. *Identifying and specifying reusable services of service centric systems through product line technology*. SOAPL Workshop, SPLC, 2007.

12. Bayer J *et al.* PuLSE: a methodology to develop software product lines. *Proceedings of the Fifth Symposium on Software Reusability (SSR99)*. ACM Press: New York, 1999.
13. Clements P, Northrop L. *Software Product Lines: Patterns and Practice*. Addison Wesley: Boston, USA, 2001.
14. Bayer J *et al.* Process family engineering: modeling variant-rich processes, 2005. (Available from: <http://publica.fraunhofer.de/starweb/servlet.starweb?path=pub.web&search=N-49329>)
15. Tsai W, Jin Z, Wang P, Wu B. Requirement engineering in service-oriented system engineering. *Proceedings of International Conference on e-Business Engineering*. IEEE, 2007.
16. Cockburn A. *Writing Effective Use Cases*. Addison Wesley, 2001.
17. Schmid K. The product line mapping approach to defining and structuring product portfolios. *Proceedings of the IEEE Joint International Conference on Requirements Engineering (RE'02)*. IEEE Computer Society, 2002.
18. Petersen K, Bramsiepe N, Pohl K. Applying variability modeling concepts to support decision making for service composition. 2nd International Workshop on Service-Oriented Computing: Consequences for Engineering Requirements (SOCCER'06), 2006.
19. Bortz J. *Statistik. Für Human- und Sozialwissenschaftler*. Springer, 2005.
20. Maiden N. *Servicing Your Requirements*. IEEE Software, September/October, 2006.
21. Sommerville I. *Integrated Requirements Engineering: A Tutorial*. IEEE Software, January/February, 2005.
22. Lam W, Jones S, Britton C. Technology transfer for reuse: a management model and process improvement framework. *Proceedings of the IEEE International Requirements Engineering Conference*. IEEE, 1998.
23. Baum L, Becker M, Geyer L, Molter G. Mapping requirements to reusable components using design spaces. *Proceedings of International Requirements Engineering Conference*. IEEE, 2000.
24. Boehm B, Abi-Antoun M, Port D, Kwan J, Lynch A. Requirements engineering, expectations management, and the two cultures. *Proceedings of the 4th IEEE International Symposium on Requirements Engineering*. IEEE, 1999.
25. Strong D, Volkoff O. *A roadmap for enterprise system implementation*. IEEE Computer, June 2004.
26. Zhu F, Tsai W. Framework-oriented analysis. *Proceedings of the 22nd International Computer Software and Applications Conference*. IEEE, 1998.
27. Keller G, Nüttgens M, Scheer A-W. *Semantische Prozeßmodellierung auf der Grundlage 'Ereignisgesteuerter Prozeßketten (EPK)'*, Universität des Saarlandes, 1992.
28. Doerr J, Hartkopf S, Kerkow D, Landmann D, Amthor P. Built-in user satisfaction—feature appraisal and prioritization with AMUSE. *Proceedings of the 15th Requirements Engineering Conference*. IEEE, 2007.
29. Arsanjani A. Service-oriented modeling and architecture—how to identify, specify, and realize services for your SOA. IBM DeveloperWorks, 2004. (Available from: <http://www-28.ibm.com/developerworks/webservices/library/ws-soa-design1>)
30. Schmid K. Planning Software Reuse—A Disciplined Scoping Approach for Software Product Lines. *PhD Thesis in Experimental Software Engineering*, Fraunhofer IRB Verlag, 2003.
31. Chen F, Li S, Chu WC-C. Feature analysis for service-oriented reengineering. *Proceedings of the 12th Asia-Pacific Software Engineering Conference (APSEC'05), Volume 00*. IEEE Computer Society, 2005.
32. Lee J, Muthig D, Naab M. Identifying and specifying reusable services of service centric systems through product line technology. *Proceedings of the First Workshop on Service-Oriented Architectures and Software Product Lines*. SEI, 2008.
33. Paech B, Kohler K. Task-driven Requirements in object-oriented development. *Perspectives on Requirements Engineering*. Kluwer Academics Publisher, 2003.
34. Recker J, Mendling J, Aalst W, Rosemann M. Model-driven enterprise systems configuration. *Proceedings of CAiSE 2006*, LNCS, vol. **4001**. Springer: Heidelberg, 2006.
35. Bayer J, Kose M, Ocampo A. Improving the development of e-business systems by introducing process-based software product lines. *Proceedings of the 7th Conference on Product Focused Software Process Improvement*. Springer: Heidelberg, 2006.
36. Scheer A-W. *ARIS—Modellierungsmethoden, Metamodelle, Anwendungen*. Springer: Heidelberg, 2001.
37. Klückmann J. *10 Schritten zur Business-Driven SOA*. IDS Scheer AG, 2007.
38. Rolland C, Kaabi R. An intentional perspective to service modeling and discovery. *Proceedings of the 31st Annual International Computer Software and Application Conference*. IEEE, 2007.
39. Alves C. COTS-based requirements engineering. *Proceedings of Component-Based Software Quality*, LNCS 2693. Springer: Heidelberg, 2003.
40. Wang J, Yu J, Falcari P, Han Y, Morisio M. An approach to domain-specific reuse in service-oriented environments. *Proceedings of the International Conference on Software Reuse*, LNCS, vol. **5030**. Springer: Heidelberg, 2008.
41. Doerr J, Adam S, Eisenbarth M, Ehresmann M. Implementing Requirements Engineering Processes: Using Cooperative Self-Assessment and Improvement. IEEE Software, May/June, 2008.
42. Simonsen J. How do we take care of strategic alignment? Constructing a design approach. *Scandinavian Journal of Information Systems* 1999; **11**: 51–72.
43. Earl MJ. Experiences in strategic information systems planning. *MIS Quarterly*. March 1993; 1–23.
44. Buchwald S, Bauer T, Reichert M. Bridging the gap between business process models and service composition specifications. *Int'l Handbook on Service Life Cycle Tools and Technologies: Methods, Trends and Advances*, 2011.

AUTHORS' BIOGRAPHIES



Sebastian Adam studied Computer Science at the Technical University of Kaiserslautern, Germany. Since 2005, he has been working at the Fraunhofer Institute Experimental Software Engineering (IESE). His research focus lies in the area of requirements engineering for information systems with a strong focus on business processes and reuse. Since 2011, he has been leading the requirements engineering group at Fraunhofer IESE.



Norman Riegel holds a diploma in Business Information Technology (University of Mannheim, Germany) and has been working as a project manager and requirements engineer for information systems at the Fraunhofer Institute Experimental Software Engineering (IESE) in Kaiserslautern, Germany, since 2007. His research interests include requirements engineering, business process management, prioritization and decision support.



Joerg Doerr is head of the Information Systems division at the Fraunhofer Institute Experimental Software Engineering (IESE) in Kaiserslautern, Germany. He was for five years head of the Requirements and Usability Engineering department. His work areas include requirements engineering, with a focus on non-functional requirements. He received his Master of Science and PhD in Computer Science from the University of Kaiserslautern, Germany. He is member of the German Computer Science Society and of the steering committee in the special interest group 'Requirements Engineering'.



Oezguer Uenal studied Computer Science at the University of Applied Sciences in Kaiserslautern, Germany. Since 2007, he has been working at the Fraunhofer Institute Experimental Software Engineering (IESE). His focus lies in requirements engineering related to information systems and business process analysis and management. He is also involved in the development and integration of new technologies within the information systems domain, such as mobile development and integration with back-end systems.



Daniel Kerkow holds a Master of Psychology from the University of Saarland, Germany, where he concentrated on media and work psychology and social cognition. He works as a scientist and consultant in the field of Requirements and Usability Engineering at the Fraunhofer Institute Experimental Software Engineering (IESE). There he is engaged in the topic of non-functional requirements, especially in the perception of product quality. Further interests include flexible and value-based processes for integrated usability and requirements engineering as well as requirements engineering for innovative products.